[Fall 2025] ROB-GY 6203 Robot Perception Homework 2

Aadhav Sivakumar

Submission Deadline (No late submission): NYC Time 5:00 PM, November 24, 2025

1. Please **start early**. Some of the problems in this homework can be **time-consuming**, both in terms of conceptual problem-solving and actual computation of the results. *It's highly likely that you will NOT be able to compute all the results if you start on the date of the deadline.*
2. Please submit the **.pdf** generated by this LaTex file. This .pdf file will be the main document for us to grade your homework. If you wrote any code, please zip all the **code** together and **submit a single .zip file**. Name the code scripts clearly and/or make explicit references in your written answers. Do NOT submit very large data files along with your code!
3. This Overleaf project is view-only. To work on this document, please use the download feature on the top left and download the source files. Then, you can re-upload them to your own Overleaf account or any other LaTeX editing tools of your preference.
4. Please typeset your report in LaTeX/Overleaf. If you have never used LaTeX/Overleaf before, the best advice is to just start typing. Try things hands-on like a true engineer! If you prefer a more structured introduction, this video can be a good help. **Do NOT submit a hand-written report!** as they will be graded zero.

   LaTeX is the standard tool for STEM academic writing due to its convenience in typesetting mathematical formulas, so this class will encourage you to get familiar with this widely used tool.
5. Do not forget to update the variables "yourName" and "yourNetID".

Contents

Task 1. RANSAC Plane Fitting (4pt)

In this task, you are supposed to fit a plane in a 3D point cloud. You have to write a custom function to implement the RAndom SAmple Consensus (RANSAC) algorithm to achieve this goal.

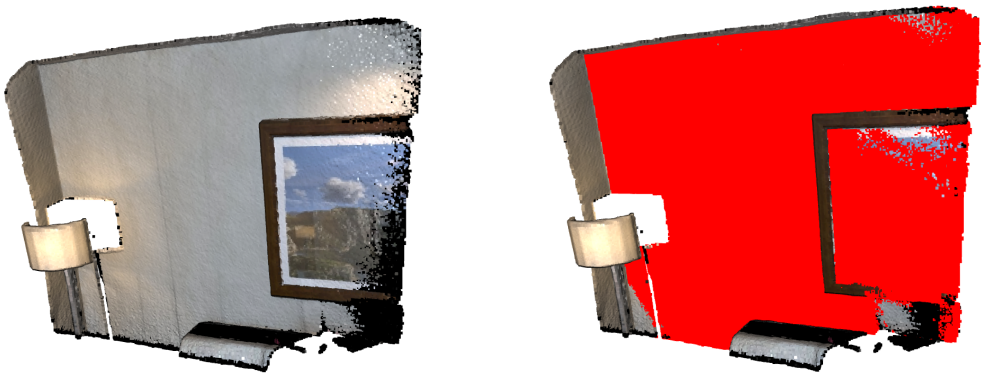


Figure 1: **Left:** Original Data, **Right:** Data with the best fit plane marked in red.

Use the following code snippet to load and visualize the demo point cloud provided by Open3D.

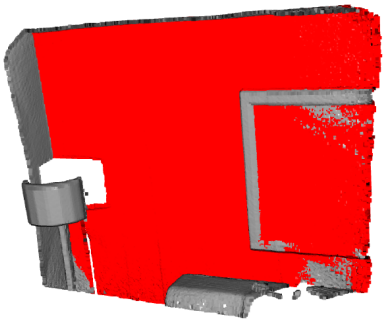
```
import open3d as o3d

# read demo point cloud provided by Open3D
pcd_point_cloud = o3d.data.PCDPointCloud()
pcd = o3d.io.read_point_cloud(pcd_point_cloud.path)

# function to visualize the point cloud
o3d.visualization.draw_geometries([pcd],
                                  zoom=1,
                                  front=[0.4257, -0.2125, -0.8795],
                                  lookat=[2.6172, 2.0475, 1.532],
                                  up=[-0.0694, -0.9768, 0.2024])
```

Note: If you use the RANSAC API in existing libraries instead of your own implementation, you will only get 60% of the total score.

Answers:
This solution manually applies the RANSAC algorithm. It does this by randomly sampling many points in the image, and creating a model fitted to that data. Then a threshold is applied to determine which are considered outliers vs inliers (basically which points can be considered as a part of the plane, versus some points that are too far off to be considered part of the plane.). I decided that the value of 0.02 worked the best and gave the best results, without including/excluding too many/few points. Here are the results:



Task 2. ICP (4pt)

In this task, you are required to align two point clouds (source and target) using the Iterative Closest Point (ICP) algorithm discussed in class. The task consists of two parts.

a) (2pt)

In part 1, you have to load the demo point clouds provided by Open3D and align them using ICP. **Caution:** These point clouds are different from the point cloud used in the previous question. You are expected to **write a custom function to implement the ICP algorithm**. Use the following code snippet to load the demo point clouds and to visualize the registration results. You will need to pass the final 4X4 homogeneous transformation (pose) matrix obtained after the ICP refinement. Explain in detail, the process you followed to perform the ICP refinement.

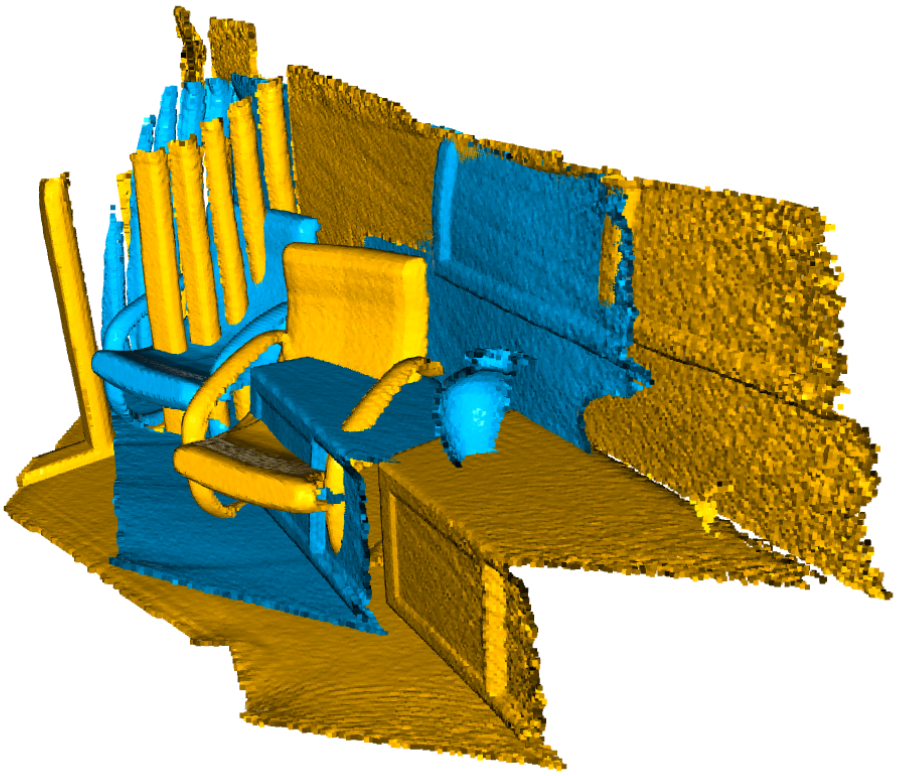
```
import open3d as o3d
import copy

demo_icp_pcds = o3d.data.DemoICPPointClouds()
source = o3d.io.read_point_cloud(demo_icp_pcds.paths[0])
target = o3d.io.read_point_cloud(demo_icp_pcds.paths[1])

# Write your code here

def draw_registration_result(source, target, transformation):
    """
    param: source - source point cloud
    param: target - target point cloud
    param: transformation - 4 X 4 homogeneous transformation matrix
    """
    source_temp = copy.deepcopy(source)
    target_temp = copy.deepcopy(target)
    source_temp.paint_uniform_color([1, 0.706, 0])
    target_temp.paint_uniform_color([0, 0.651, 0.929])
    source_temp.transform(transformation)
    o3d.visualization.draw_geometries([source_temp, target_temp],
                                      zoom=0.4459,
                                      front=[0.9288, -0.2951, -0.2242],
                                      lookat=[1.6784, 2.0612, 1.4451],
                                      up=[-0.3402, -0.9189, -0.1996])
```

Answers:



```
Final Transformation Matrix:
[[ 0.8296498   0.00508072 -0.55826105  1.35787875]
 [-0.20635645  0.93193285 -0.29819151  1.06359555]
 [ 0.51874678  0.36259529  0.77422634 -1.41446555]
 [ 0.          0.          0.          1.        ]]
```

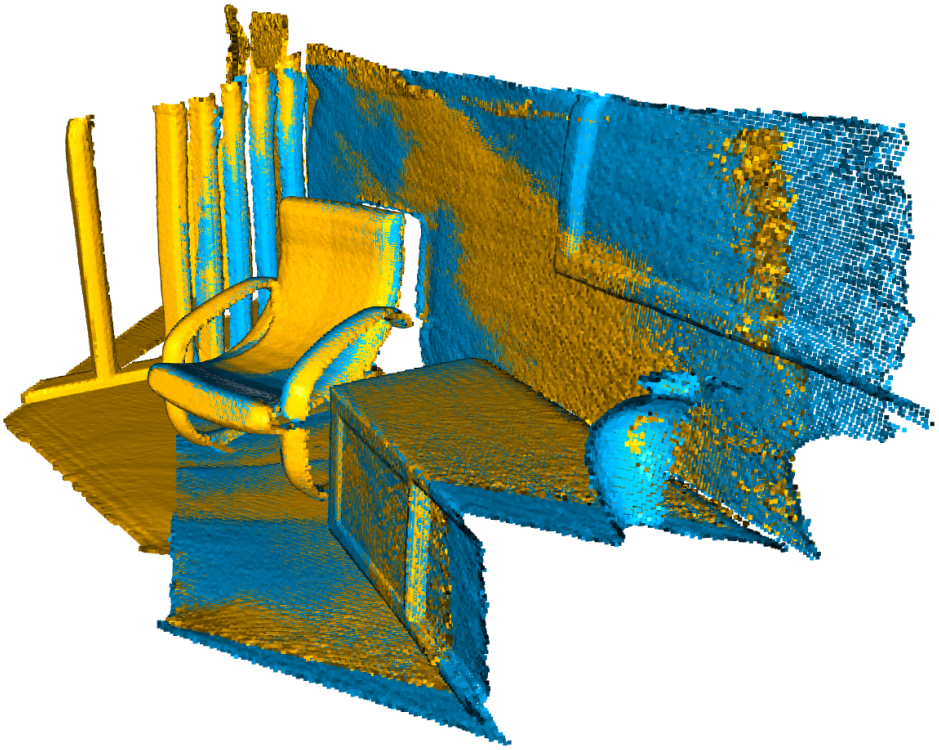
It starts with the identity matrix, and it's slowly modified to create the final "best fitting" one. For each point, you calculate the closest point in the target cloud. You can then calculate the centroid, sum them all up, and then adjust the transformation matrix to make it lower. You continue this process through a couple iterations, until you either hit the maximum number of iterations or you go below the threshold of differences.

b) (2pt)

You have been given two point clouds from the KITTI dataset, one of the benchmark datasets used in the self-driving domain. *The point clouds are located in the data/Task2 folder under the overleaf project: https://www.overleaf.com/read/fzhnnqsqrnbz*. Repeat part 1 using these two point clouds. Compare the results from part 1 with the results from part 2. Are the point clouds in part 2 aligning properly? If no, explain why. Provide the visualizations for both parts in your answer.

Note: If you use an ICP API in existing libraries instead of your own implementation, you will only get 60% of the total score.

Answers:



Task 3. F-matrix and Relative Pose (4pt)

a) (2pt)

Use your own camera, take two pictures, and **estimate** the fundamental matrix induced by these pictures.

To perform a good estimation, it is recommended that you take pictures in a manner similar to the sketching in Fig. 2. In the figure, $\mathbf{x}$ and $\mathbf{x}'$ can be thought of as two pictures, with $\mathbf{c}$ and $\mathbf{c}'$ representing the camera centers of the camera that took the pictures.

Fig. 3 represents a pair of pictures that can be used to estimate the fundamental matrix easily.

Tip: This problem can reuse something you have done in HW1.

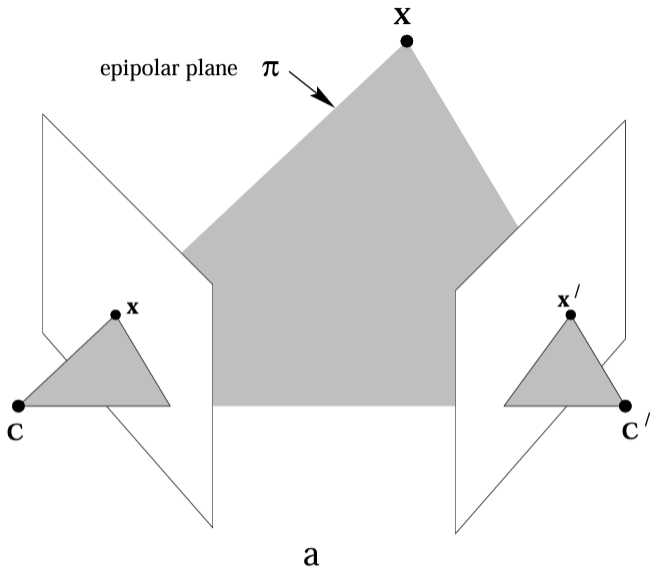


Figure 2: Epipolar Geometry

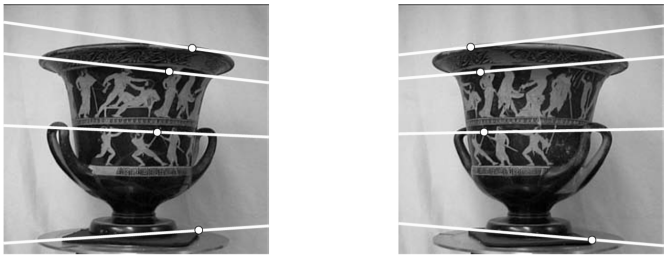


Figure 3: Caption

Answers:

```
Estimated Fundamental Matrix F:
[[ 4.13059060e-09  4.29625912e-08 -9.81003260e-05]
 [ 8.23583662e-08  1.20512046e-08  8.41036342e-04]
 [-2.70485965e-04 -1.17557346e-03  1.00000000e+00]]
```

b) (1pt)

Draw epipolar lines in both images. You don't need to explain the process. Just provide the visualization.

Answers:

Name: Aadhav Sivakumar, netID: as20436

c) (1pt)

Find the relative pose (R and t) between the two images, expressed in the left image's frame. Before you give the solution, answer the following two questions.

1. Can you directly use the F-matrix you estimated in a) to acquire R and t without calculating any other quantity?
2. If yes, please describe the process. If no, what other quantity/matrix do you need to calculate to solve this problem? Please calculate that information and proceed to find the relative pose. Does the estimated relative pose make sense to you, given that you are the one performing camera movement?

Answers:
1. No, because the fundemental matrix encodes this data into its values. There are also no measurements, meaning that an object twice as big could occupy the same amount of space in the frame, but at a particular distance it would look the same, meaning it can't be retrieved mathematically

2. We would need the camera intrinsic matrix K, as well as the essential matrix E.

```
Assumed Camera Intrinsic Matrix K:
[[5.712e+03 0.000e+00 2.142e+03]
 [0.000e+00 5.712e+03 2.856e+03]
 [0.000e+00 0.000e+00 1.000e+00]]

Essential Matrix E:
[[ 0.13476855  1.40173806  0.19105817]
 [ 2.6871018   0.39319398  6.00825975]
 [-0.15092673 -5.99262684  0.13895795]]

Recovered Rotation Matrix R:
[[ 0.79782056 -0.00353448 -0.60288461]
 [-0.01637638  0.99948679 -0.02753111]
 [ 0.60267252  0.03183795  0.79735324]]

Rotation angle: 37.12 degrees

Recovered Translation t (up to scale):
[ 0.97360324 -0.036172    0.22536262]
Normalized direction: [ 0.97360324 -0.036172    0.22536262]
```

Task 4. Object Tracking (4pt)

Given a short video sequence, persistently track the entities in said sequence across time until it goes out of frame, or the sequence terminates. You can find the aforementioned sequence in the *data/Task4 folder of this Overleaf project*. You are free to use any tracking algorithm or pre-trained model for the task. With your implementation, answer the following two questions:

1. Can you explain in detail, the process or algorithm you used to perform the tracking?

2. Additionally, can you provide a few example frames from your resulting sequence with the proper visualization to demonstrate the efficacy of your implementation?

Note: By *tracking*, it means that you should be able to return the bounding box or centroid coordinate(s) of the entities in the video across the entire sequence.

Tip 1: For the second part of the question, you should provide an example via a few adjacent frames so that the tracking performance is obvious. You should also use consistent labelling or color coding for your visualization, corresponding to each unique entity, for consistency if possible.

Tip 2: You should yield something like the following for your implementation and submission.

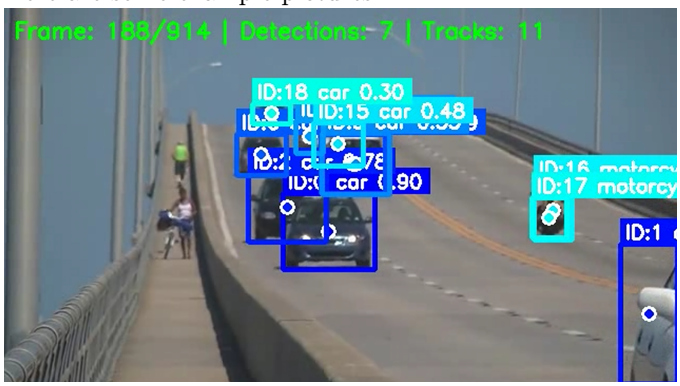
Figure 4: Frame t_1

Figure 5: Frame t_2

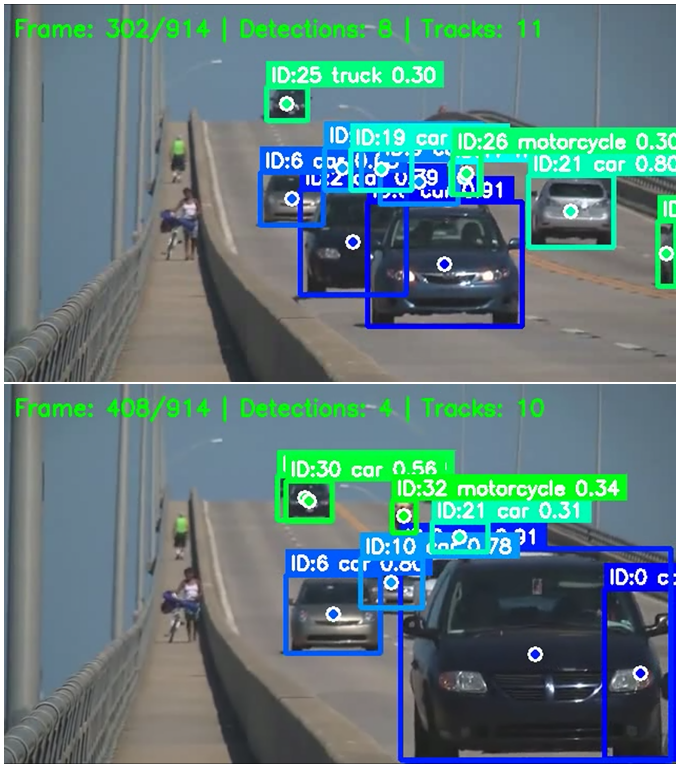
Answers:
1. I decided to use YOLO for this part, which is a very popular object tracking framework. It seems like it is currently maintained by ultralytics. It works by passing the images through an existing model, that is a neural network with 24 layers. They are constantly retraining and improving the models, and are currently on the 11th version.

It works by splitting the image into blocks, so it doesn't have to individually process every single pixel. It then runs a very small model to determine if anything significant is happening in that block. If it's mostly the same color, it determines that there are probably no objects in that section. It then uses Intersection Over Unions to determine if there are objects that go into multiple blocks. At this point, it has a rectangle that contains an exact object of interest, and it can now run it through the bigger model to determine what it is.

Here are some example pictures:



Name: Aadhav Sivakumar, netID: as20436


Frame: 302/914 | Detections: 8 | Tracks: 11
ID:25 truck 0.30
ID:26 motorcycle 0.30
ID:21 car 0.80
Frame: 406/914 | Detections: 4 | Tracks: 10
ID:32 motorcycle 0.34
ID:21 car 0.31

Task 5. Skiptrace (4pt)

a) (3pt)

Find the three pictures in the database folder that most closely resemble the query pictures.

It is very possible that you may find pictures that, while not the intended solution to this problem, still very closely resemble the query images. To avoid confusion, I marked the correct database images with something you cannot miss.

Upload the query results from `query_1.jpg`, `query_2.jpg`, and `query_3.jpg`.

Note that there is also a `query_1_1.jpg`. You do not need to do anything with it for now.

Tip 1: This question can be time-consuming and memory-intensive given the data you need to process. Please start early.

Tip 2: Refrain from using `np.vstack()/np.hstack()/np.concatenate()` too often. A numpy array is designed in a way that frequently resizing it will be very time-consuming/memory-intensive. Consider other options in Python if the need to concatenate arrays arises.

Answers:

a)

```
Task 5a - Top-1 Matches:
  query_1.jpg → dr5rsn0y7mp2-dr5rsn0yk3p6-cds-3670f40ef7987d5a-20161022-1119-812.jpg
  query_2.jpg → dr5rsnxr62tk-dr5rsnxr5kr5-cds-1145ca086c7f4f26-20161202-1520-158.jpg
  query_3.jpg → dr5rsp3yhyug-dr5rsp3yhmb6-cds-b082282c12fc5800-20161105-1549-2571.jpg
```

b) (1pt)

Perform a query on the `query_1_1.jpg` picture. This time, instead of having the top-1 similar result, upload the top-5 similar pictures.

Also, explain how you define "similar" mathematically.

Answers:

b)

```
Task 5b - Top-5 Matches for query_1_1.jpg:
  1. dr5rsn0y7w4d-dr5rsn0ykd42-cds-3670f40ef7987d5a-20161022-1109-7894.jpg
  2. dr5rsn0y7tfr-dr5rsn0ykd42-cds-3670f40ef7987d5a-20161022-1109-7953.jpg
  3. dr5rsn0y7w47-dr5rsn0ykd5n-cds-3670f40ef7987d5a-20161022-1109-7804.jpg
  4. dr5rsn0y7wcq-dr5rsn0ykdsd-cds-3670f40ef7987d5a-20161022-1109-5475.jpg
  5. dr5rsn0y7kzq-dr5rsn0yk2ys-cds-3670f40ef7987d5a-20161022-1119-901.jpg
```

"similarity" is defined by a combination of color distributions and feature matches